



CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

Profa. Rachel Reis
rachel@inf.ufpr.br

AULA DE HOJE

Permutação e Combinação com Recursão

Adaptação do material gentilmente
disponibilizado pelos Profs. André Vignatti e
Daniel Oliveira

MOTIVAÇÃO

- Alguns padrões aparecem de forma recorrente para solucionar diversos problemas computacionais.
- Essas soluções exigem que tais padrões sejam gerados de forma metódica para percorrer o espaço de busca.
- Exemplos de padrões : permutação e combinação.

PERMUTAÇÃO

- Agrupamentos formados por n elementos, de forma que a ordem desses elementos seja sempre diferente.
- Sobre a permutação...
 - Cada agrupamento é formado por n elementos.
 - A ordem dos elementos é relevante.
- Exemplo (ABC):
 - “ABC”, “ACB”, “BAC”, “BCA”, “CBA”, “CAB”

PERMUTAÇÃO

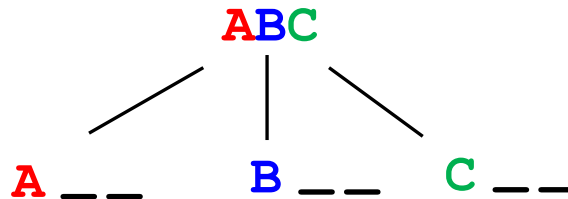
- Problema computacional

Permutação de strings
Instância: s , onde s é uma string de tamanho n que não possui elementos (caracteres) repetidos.
Resposta: todas as permutações de s com tamanho n .

PERMUTAÇÃO

- Como abordar o problema da permutação de strings usando recursão?
 1. Para cada caractere **c** da string **s**, colocar **c** no início
 - 1.1 Gerar todas as permutações dos caracteres restantes (recursão com $n - 1$ caracteres)

- Exemplo:



PERMUTAÇÃO

- Função *wrapper*

listar_permutacoes (<i>string</i>)
<code>permutar</code> (" ", <i>string</i>)

- Algoritmo recursivo

<code>permutar</code> (<i>pronto</i> , <i>restante</i>)
Se <i>restante</i> é vazio imprime(<i>pronto</i>) Senão Para todo caractere <i>c</i> em <i>restante</i> <code>permutar</code> (<i>pronto</i> + <i>c</i> , <i>restante</i> - <i>c</i>)

PERMUTAÇÃO

```
permutar(pronto, restante)
```

Se *restante* é vazio

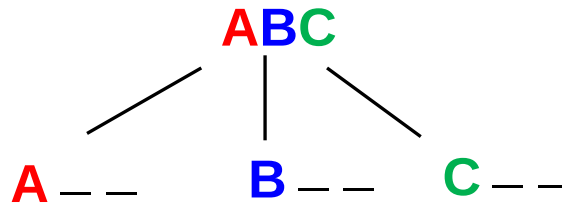
imprime(*pronto*)

Senão

Para todo caractere *c* em *restante*

permutar(*pronto* + *c*, *restante* - *c*)

- Sobre o algoritmo recursivo
 - As entradas *pronto* e *restante* são strings.
 - A entrada *pronto* inicia como uma string vazia e a entrada *restante* com a string a ser permutada.
 - A cada iteração do laço **Para** na string original, o algoritmo pega um caractere da **string restante** e coloca no início.



PERMUTAÇÃO

- Algoritmo recursivo

Caso base



permutar(*pronto*, *restante*)

Se *restante* é vazio
imprime(*pronto*)

Senão

Para todo caractere *c* em *restante*

Recursão



permutar(*pronto* + *c*, *restante* - *c*)

PERMUTAÇÃO

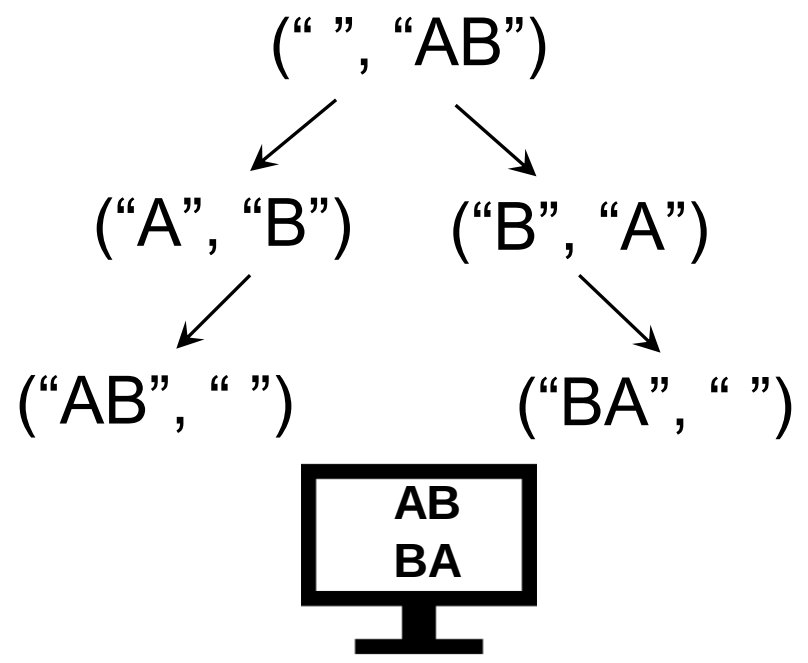
- Como ficaria a saída do algoritmo recursivo permutar para a string AB?

permutar(<i>pronto</i> , <i>restante</i>)
Se <i>restante</i> é vazio imprime(<i>pronto</i>) Senão Para todo caractere <i>c</i> em <i>restante</i> permutar(<i>pronto</i> + <i>c</i> , <i>restante</i> - <i>c</i>)

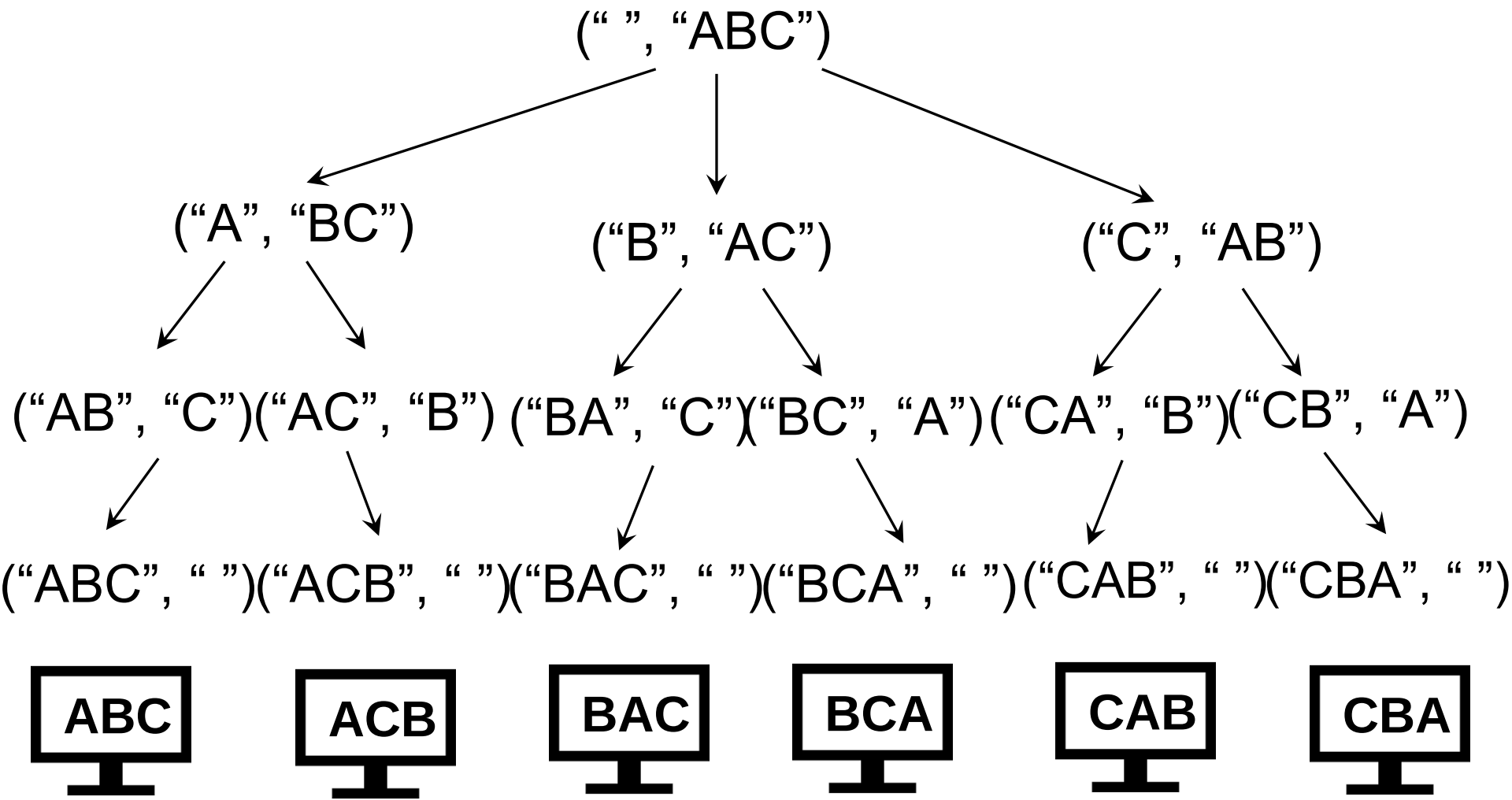
Ex1. Executar para a string AB

```
permutar(" ", "AB")
  c → A
    permutar("A", "B")
      c → B
        permutar("AB", " ")
          imprime(AB)
      c → B
        permutar("B", "A")
          c → A
            permutar("BA", " ")
              imprime(BA)
```

permutar(<i>pronto</i> , <i>restante</i>)
Se <i>restante</i> é vazio imprime(<i>pronto</i>)
Senão Para todo caractere <i>c</i> em <i>restante</i> permutar(<i>pronto</i> + <i>c</i> , <i>restante</i> - <i>c</i>)



Ex2. Executar o algoritmo permutar para a string ABC



PERMUTAÇÃO

- Exemplo de aplicação
 - Caixeiro viajante: um vendedor precisa visitar várias cidades, passando por cada uma apenas uma vez, e voltar à cidade inicial com o menor custo total.

PARA PRATICAR...

- Monte a árvore de execução do algoritmo *permutar(pronto, restante)* para a string ABCD.

COMBINAÇÃO

- Tipo de agrupamento que calcula quantos subconjuntos de x elementos pode-se formar de um conjunto inicial com n elementos ($x \leq n$).
- Sobre a combinação...
 - Os subconjuntos pode ter tamanhos diferentes.
 - A ordem entre os elementos não importa ($AB = BA$).
- Exemplo de subconjuntos de ABC:
 - “ABC”, “AB”, “AC”, “A”, “BC”, “B”, “C”, “ ”

COMBINAÇÃO

- Problema computacional

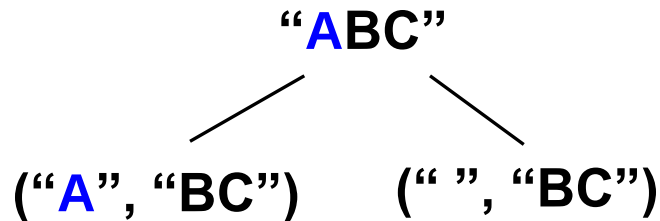
subconjuntos de uma string
Instância: s , onde s é uma string de tamanho n .
Resposta: todos os subconjuntos dos caracteres de s .

COMBINAÇÃO

- Como abordar o problema dos subconjuntos de uma string usando recursão?

1. Dado um caractere *c* da string *s*, vamos colocá-lo ou não no conjunto

Chamadas recursivas { 1.1 Gerar todos os subconjuntos de caracteres **incluindo** *c*
1.2 Gerar todos os subconjuntos de caracteres **ignorando** *c*



COMBINAÇÃO

- Função *wrapper*

listar_subconjuntos(<i>string</i>)

<i>subconjuntos</i> (" ", <i>string</i>)

- Algoritmo recursivo

<i>subconjuntos</i> (<i>pronto</i> , <i>restante</i>)

Se *restante* é vazio

 imprime(*pronto*)

Senão

$c \leftarrow \text{restante}[1]$

subconjuntos(*pronto* + *c*, *restante* - *c*)

subconjuntos(*pronto*, *restante* - *c*)

COMBINAÇÃO

subconjuntos(*pronto*, *restante*)

Se *restante* é vazio

imprime(*pronto*)

Senão

$c \leftarrow \text{restante}[1]$

subconjuntos(*pronto* + *c*, *restante* - *c*)

subconjuntos(*pronto*, *restante* - *c*)

- Sobre o algoritmo recursivo...
 - As duas entradas, *pronto* e *restante*, são strings.
 - A primeira é o subconjunto *pronto* que inicia como uma string vazia e a outra que é a entrada *restante* com os caracteres que faltam analisar.

COMBINAÇÃO

- Como ficaria a saída do algoritmo recursivo subconjuntos para a string AB?

subconjuntos(*pronto*, *restante*)

Se *restante* é vazio

 imprime(*pronto*)

Senão

$c \leftarrow \text{restante}[1]$

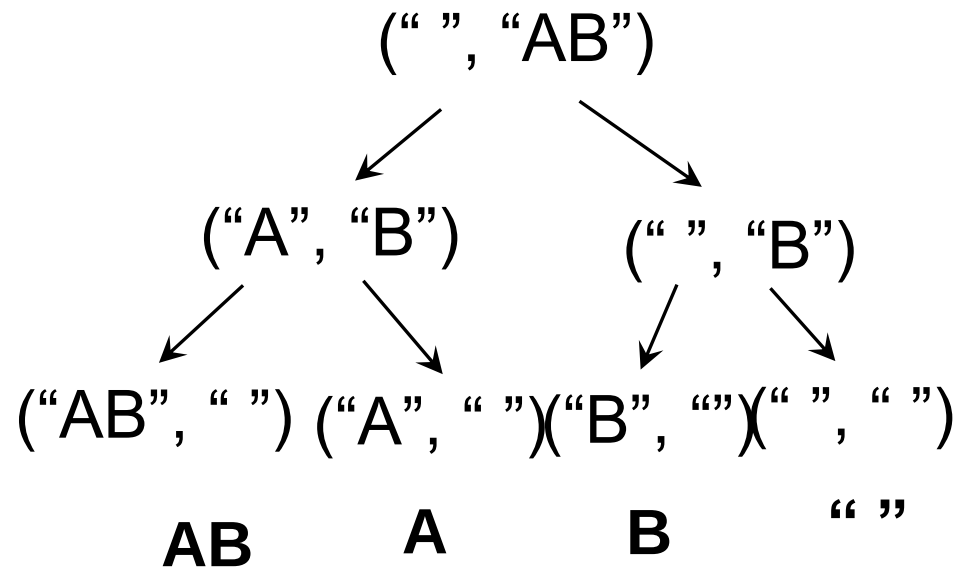
 subconjuntos(*pronto* + *c*, *restante* - *c*)

 subconjuntos(*pronto*, *restante* - *c*)

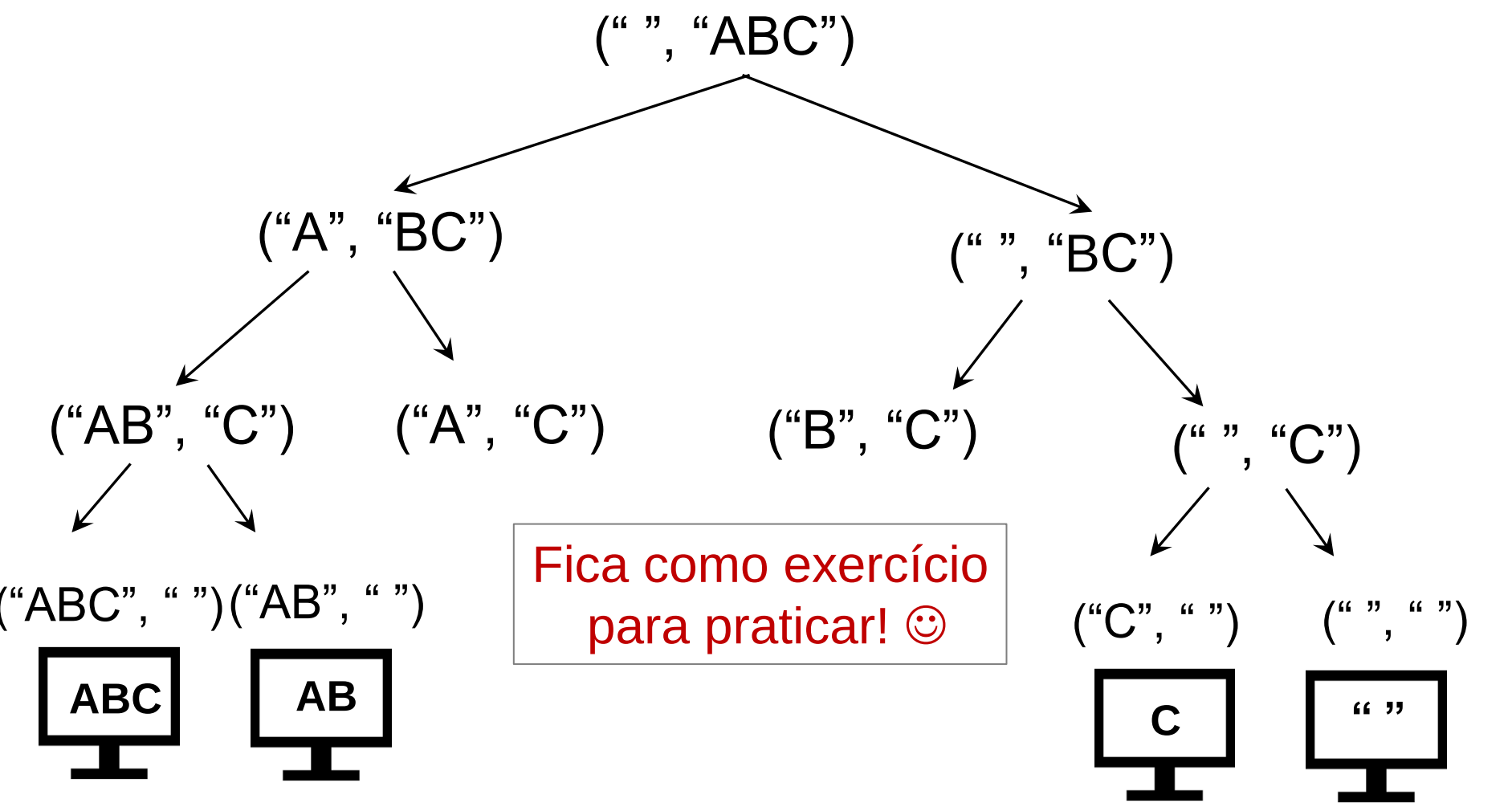
Ex1. Executar para a string AB

```
subconjuntos("", "AB")
  c → A
  subconjuntos("A", "B")
    c → B
    subconjuntos("AB", "")
    subconjuntos("A", "")
  subconjuntos("", "B")
    c → B
    subconjuntos("B", "")
    subconjuntos("", "")
```

subconjuntos(<i>pronto</i> , <i>restante</i>)
Se <i>restante</i> é vazio imprime(<i>pronto</i>)
Senão <i>c</i> ← <i>restante</i> [1] subconjuntos(<i>pronto</i> + <i>c</i> , <i>restante</i> - <i>c</i>) subconjuntos(<i>pronto</i> , <i>restante</i> - <i>c</i>)



Ex2. Executar o algoritmo subconjuntos para a string ABC



COMBINAÇÃO

- Exemplo de aplicação
 - Mala de livros: preencher uma mala com o maior número de livros com diferentes pesos, sem ultrapassar o limite máximo de peso da mala (ex.: 23Kg).

PARA PRATICAR...

- Monte a árvore de execução do algoritmo *subconjuntos(pronto, restante)* para a string ABCD.

CONCLUSÃO

- “Esqueleto” dos algoritmos de permutação e combinação.
- Podemos fazer variações para chegar no padrão que queremos.
 - Ex.: permitir elementos repetidos.